



Nombre y Apellido:

Legajo:

Examen Final

1. Definimos el **tad** *Matriz*, con las siguientes operaciones:

```
tad Matriz (V : Ordered Set) where
import Bool, Nat, Pares
  new      : Nat → Nat → V → Matriz V           -- new n m v crea una matriz de n x m
                                                    -- con sus valores inicializados con v
  set      : Nat → Nat → V → Matriz V → Matriz V -- set i j v m sustituye el valor de la
                                                    -- celda (i,j) en m por v
  elem     : Nat → Nat → Matriz V → Matriz V       -- elem i j m devuelve el valor de la celda (i,j) en m
  dimension : Matriz V → (Nat, Nat)
  transpose : Matriz V → Matriz V                 -- calcula la transpuesta de una matriz
```

a) Dar la especificación algebraica del **tad** *Matriz*.

b) Se desea implementar el **tad** en Haskell utilizando el siguiente tipo de datos para representar matrices:

```
type Vector a = [a]
type Matriz a = [Vector a]
```

- Implementar las operaciones *elem* y *dimension* del **tad**.
- Definir las funciones *fila* :: *Int* → *Matriz a* → *Vector a* y *columna* :: *Int* → *Matriz a* → *Vector a*, que calculen la *i*-ésima fila o columna de una matriz.
- Implementar la función *transpose* del **tad**.

2. Para resolver las finanzas de un país un presidente necesita saber hasta qué fecha puede pagar los intereses de una deuda si cuenta con \$*X*.

Usando las funciones del **tad** *secuencia*, definir una función *pagarDeuda* :: *Seq* (*Float*, *Fecha*) → *Float* → *Maybe Fecha*, que dada una secuencia con los valores en pesos de los intereses que debe pagar el país en las fechas dadas y un monto de pesos *X*, calcule la fecha en que el dinero *X* se termine si se pagan los intereses anteriores a esa fecha. Por ejemplo,

pagarDeuda ((15, *f*₁), (20, *f*₂), (25, *f*₃), (35, *f*₄), (50, *f*₅), (70, *f*₆)) 80 = *f*₃

Definir *pagarDeuda* usando la función *scan*, con profundidad en $O(\lg n)$ y trabajo en $O(n)$, donde *n* es largo de la secuencia.

3. Se representan secuencias mediante árboles binarios dados por el siguiente tipo de datos:

```
data Tree a = E | L a | N (Tree a) (Tree a)
```

donde el recorrido *inorder* del árbol da el orden de los elementos de la secuencia. Definir en Haskell, de manera eficiente, la función *concatMap* :: (*a* → *Tree b*) → *Tree a* → *Tree b*, que aplica una función a todos los elementos de una secuencia y concatena las secuencias resultantes. Por ejemplo,

concatMap ($\lambda n \rightarrow \text{replicate } n \ n$) <1, 2, 3> = <1, 2, 2, 3, 3, 3>